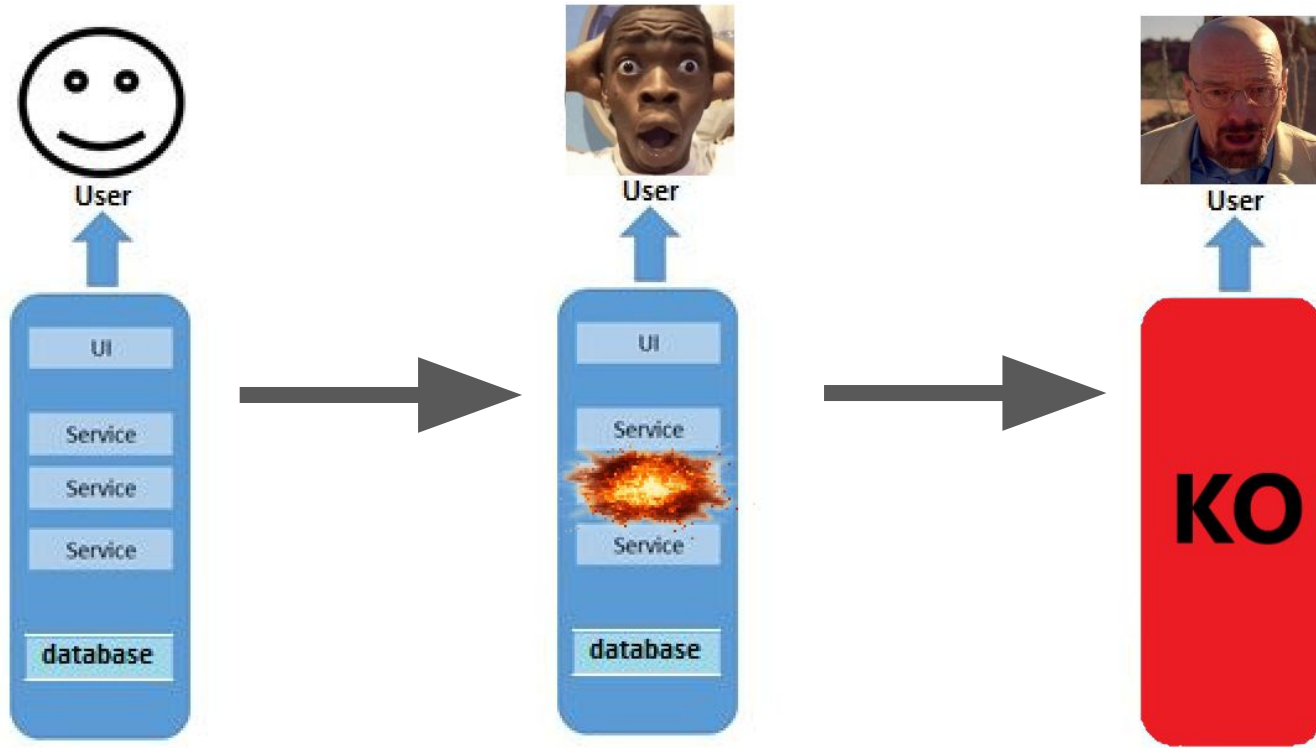


Introduction aux microservices



Le problème du design monolithique



Le problème du design monolithique

Comme dans le cas de Netflix, les apps monolithiques peuvent être très efficaces jusqu'à ce qu'elles soient trop volumineuses et que la mise à l'échelle se complique. Apporter un petit changement à une seule fonction nécessite de compiler et de tester l'ensemble de la plateforme, ce qui va à l'encontre de la haute disponibilité requise pour une telle application.

Cela pose aussi plusieurs problèmes divers, parmi lesquels :

- **Stabilité** - si une erreur survient dans un module, elle peut affecter la disponibilité de l'ensemble de l'app.
- **Evolution** - un monolithe est limité par les technologies déjà utilisées en son sein. L'adaptation de nouveaux framework ou de langage affecte l'ensemble de l'app et est par conséquent très chronophage.
- **Vitesse de développement** - une app monolithique volumineuse complexifie et ralentit le développement. Isoler un problème lors d'un processus de calcul lourd peut s'avérer complexe.



La solution

Il existe un certain nombre de définition des microservices. La plus commune et généralisée reste celle de Martin Fowler qui dit : *“Le style architectural des microservices est une approche permettant de développer une application unique sous la forme d’une suite logicielle intégrant plusieurs services. Ces services sont construits autour des capacités de l’entreprise et peuvent être déployés de façon indépendante.”*

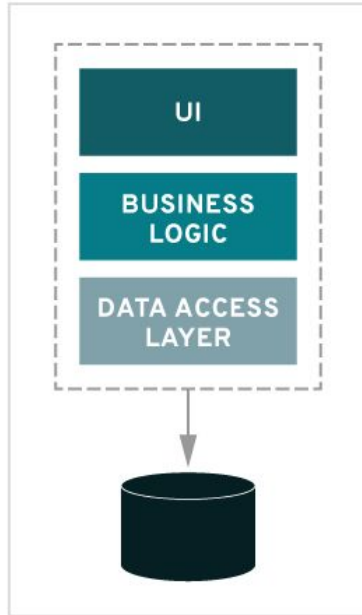
Concrètement, les microservices sont une méthode développement logiciel utilisée pour concevoir une application comme un ensemble de services modulaires. Chaque module répond à un objectif métier spécifique et communique avec les autres modules.

Cette architecture permet à un projet d’avoir ses “briques” de services développés et déployé indépendamment les uns des autres. Cela permet de faire évoluer une application sans en re-déployer l’intégralité, et facilite l’identification des problèmes. Les microservices étant indépendants, lorsqu’un élément tombe en panne ou rencontre un problème, l’ensemble de l’application ne cesse pas de fonctionner contrairement aux applications monolithiques.



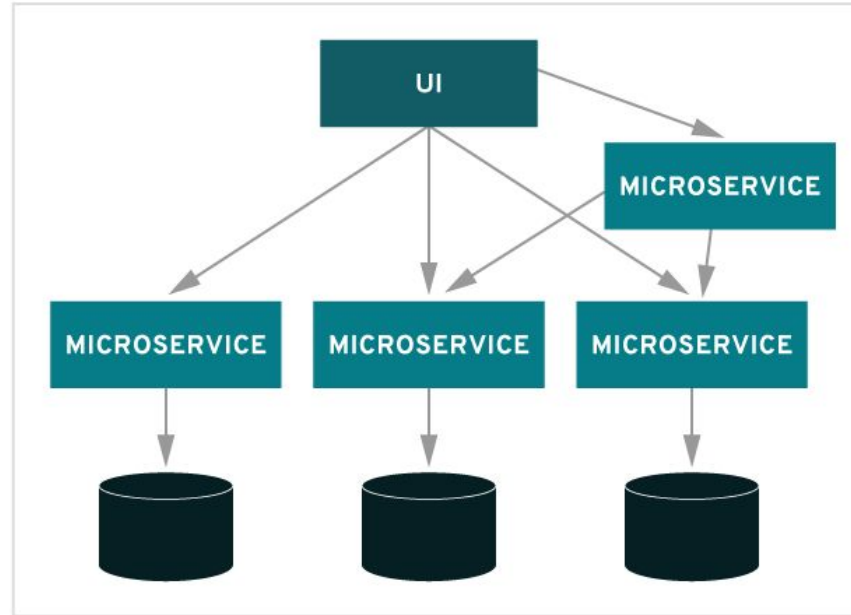
La solution ...

MONOLITHIC



VS.

MICROSERVICES



a quand même ses limites !

Tandis que les principes de l'Architecture Micro-Services sont généralement partagés par tous, car ils constituent autant d'avantages, les inconvénients sont plus rarement évoqués. Ces inconvénients sont bien souvent les conséquences de ces principes et constituent parfois des contradictions. Il est toutefois possible d'atténuer ces inconvénients : c'est en fait là que réside le talent de l'architecte.

- Problème de mise à jour des bases de données cloisonnées (chaque micro-service ayant sa propre base et pouvant être instancié plusieurs fois par le mécanisme de mise à l'échelle).
- Défi en matière de débogage, test, déploiement des applications constituées de micro-services.
- Changements compliqués à cause des éventuelles dépendances entre les micro-services. Les micro-services supportent toutefois le versioning et plusieurs versions d'un même micro-service peuvent coexister, permettant une migration progressive vers la dernière version.
- Application globale moins performante (latence), car dépendante du réseau (éventuellement moins fiable). Un protocole de communication asynchrone sera souvent préféré à un protocole synchrone, afin d'éviter d'attendre trop longtemps une réponse « immédiate » à chaque requête. De plus, l'usage d'un mécanisme de lecture via une mémoire cache est recommandé, afin d'optimiser la performance.
- Besoin d'authentification, voire de chiffrement, pour diminuer les failles de sécurité du réseau dans les communications interservices.

